Smart University Assistant System

Presented By :

Abdllah Atef Abdelwanis Isaa
Ahmed Yahia Kamal Mohamed
Nariman Boutros Samir Shawqy
Youssef Mohamed Seifallah

Supervised by:
DR/Azza Abd-El Rahman

2025/2026

Table of Contents

Project Overview

This project presents a complete smart academic management system that consists of four main components: Backend, AI module, Dashboard, and a mobile application developed using Flutter.
The system supports managing academic operations such as users, courses, assignments, quizzes, attendance, grades, and university events through a centralized backend built using clean architecture principles.
The AI module analyzes student performance using a weighted scoring algorithm and rule-based classification to detect at-risk students and estimate failure probability.

The dashboard provides an interactive interface for administrators to monitor performance, visualize data, and access AI insights.
Additionally, a mobile application is developed using Flutter to provide students and doctors with a user-friendly interface to interact with the system, including quizzes, assignments, attendance tracking, results, and course management.
Overall, the system demonstrates how modern technologies and simple AI techniques can be integrated into a scalable and real-world academic solution.

Smart

Academic

System

Objectives

- Build a complete academic management system
- Develop a scalable backend architecture
- Integrate AI for student performance analysis
- Create an interactive dashboard
- Develop a Flutter mobile application
- Support different user roles
- Improve academic decision-making
- Reduce manual work and increase efficiency

Smart

Academic

System

Problem & Solution

Problem

- Traditional academic systems only store data without analysis
- No early detection of students at risk
- Difficulty in tracking student performance manually
- Lack of clear insights for decision-making
- Data is scattered and not centralized
- No intelligent support for improving student performance
- Difficulty in finding specific data quickly
- Information overload when viewing large datasets
- Time-consuming manual search in tables and reports
- No ability to focus on specific categories (e.g., high-risk students)

Solution

- Centralized system to manage all academic operations
- AI-based analysis to evaluate student performance
- Early detection of at-risk students
- Clear dashboard for data visualization
- Automated tracking of attendance, assignments, and quizzes
- Smart recommendations to help students improve
- Advanced filtering system to quickly access specific data
- Ability to filter students by risk level, course, or performance
- Reduced data overload by displaying only relevant information
- Faster and more efficient decision-making

Target Audience

Students

- Track grades and academic performance
- Access quizzes, assignments, and schedules
- Receive AI-based academic risk analysis
- Improve learning and organization

Doctors / Faculty Members

- Manage courses, quizzes, and assignments
- Monitor student performance and attendance
- Publish grades and academic reports
- Identify at-risk students early

University Administration

- Manage users and academic workflows
- Monitor system-wide performance
- Improve communication between students and faculty
- Support digital transformation in education

User Guide

Admin Role

The Admin is responsible for managing the entire system and controlling academic structure.

- Admin can log in to the system
- Admin can add faculties
- Admin can add students and doctors
- Admin can create and manage courses
- Admin can enroll students in courses
- Admin can publish students' results
- Admin can view AI academic performance analysis

Student Role

The Student uses the platform to manage their learning activities and track academic progress.

- Student can log in to the portal
- Student can view enrolled courses
- Student can submit assignments
- Student can take quizzes
- Student can view published results
- Student can access AI academic analysis

Doctor Role

The Doctor manages academic content, assessments, and student evaluation.

- Doctor can log in to the system
- Doctor can create quizzes and assignments
- Doctor can upload lectures and materials
- Doctor can record student attendance
- Doctor can enter and manage student grades

System Architecture

About System

The system consists of four main components:

Backend
(ASP.NET Core
Web API)

AI Module
(Python / FastAPI
logic)

Dashboard
(HTML, Tailwind
CSS, JavaScript)

Flutter
JWT (JSON Web Token)
RESTful APIs
JSON Serialization

Implementation Overview

The system was implemented using ASP.NET Core Web API and Flutter following Clean Architecture and Domain-Driven Design principles.

The backend was divided into:

- Entities
- Repositories
- Services
- Controllers

This structure improves:

- Scalability
- Maintainability
- Separation of concerns
- Code organization

Entity Framework Core was used for database management and migrations.

JWT Authentication was implemented for secure login and role-based authorization for:

- Admins
- Students
- Doctors

The frontend dashboard and mobile application were integrated with REST APIs using Dio for efficient communication between frontend and backend systems.

Flutter Riverpod was used for reactive state management and optimized UI updates.

The project also integrated an AI academic analysis module using Python FastAPI to evaluate student performance, classify academic risk levels, and provide intelligent educational support.

Architecture Layers

The system follows a layered architecture based on Clean Architecture and Domain-Driven Design (DDD) principles to ensure scalability, maintainability, and clear separation of concerns.

The project is divided into the following layers:

- Presentation Layer (Frontend & Mobile App)
- Handles user interfaces and user interactions using Flutter, HTML, Tailwind CSS, and JavaScript.
- API Layer (Controllers)
- Exposes RESTful APIs and handles HTTP requests and responses.
- Application Layer (Services & DTOs)
- Contains business logic, application services, validations, and data transfer objects.
- Domain Layer (Entities & Interfaces)
- Includes core business entities, rules, and repository interfaces.
- Infrastructure Layer (Repositories & Database)
- Responsible for database operations, repositories, and external services integration.
- AI Service Layer (FastAPI)
- Handles academic performance analysis and student risk prediction.

Frontend & Mobile Application

The frontend and mobile application were developed using:

- Flutter & Dart for cross-platform mobile and desktop support
- HTML, Tailwind CSS, and JavaScript for web dashboard interfaces
- Responsive UI design for different screen sizes
- Reusable UI components for better maintainability
- REST API integration for dynamic data handling

The application supports:

- Android
- Web
- Desktop platforms

Backend

The backend was developed using ASP.NET Core Web API.

It is responsible for:

- Authentication & Authorization
- Business Logic
- Course Management
- Quiz & Assignment Management
- Attendance Tracking
- Results & Grades Management
- API Communication
- Role-Based Access Control

The backend follows:

- Clean Architecture
- Domain-Driven Design (DDD)
- Repository Pattern
- Dependency Injection

Database

The system uses SQL Server as the main database.
Entity Framework Core was used as the ORM for:

- Database operations
- Migrations
- Relationships handling
- Query management

The database stores:

- Users
- Courses
- Assignments
- Quizzes
- Attendance records
- Grades and results

AI Module

The AI module was developed using Python FastAPI.

It performs:

- Student academic performance analysis
- Risk level classification
- Failure probability estimation
- Personalized academic recommendations

The AI logic is based on:

- Weighted scoring algorithms
- Rule-based classification
- Academic data analysis

Technologies & Tools Used

1 Backend:

- ASP.NET Core Web API
- Entity Framework Core
- SQL Server

2 AI:

- Python (FastAPI)
- Pydantic for data validation
- Rule-Based classifaction algorithm
- weighted average algorithm

3 Frontend:

- HTML, CSS, Tailwind
- JavaScript

4 Flutter:

- * Dart
- * Riverpod
- * Dio
- * Flutter ScreenUtil
- * JWT Decoder
- * Intl Package

Backend Architecture & Implementation

Domain Layer

- Contains entities (User, Course, StudentGrade)
- Business rules and logic
- Independent from frameworks

Application Layer

- Services (UserService, GradeService)
- DTOs and validation
- Handles business use cases

Infrastructure Layer

- Database access using Entity Framework Core
- Repository implementations

Presentation Layer (API)

- RESTful APIs using controllers
- Handles HTTP requests and responses

Backend Responsibilities

User management

Course management

Quiz & assignment management

Attendance tracking

API handling

Authentication & authorization

Backend Architecture & Implementation

Backend Architecture

- Clean Architecture implementation
- Domain-Driven Design (DDD) principles
- Layered architecture for separation of concerns
- Scalable and maintainable backend structure
- Built using ASP.NET Core Web API

Authentication & Authorization

- JWT-based authentication
- Role-Based Authorization
- Admin, Doctor, and Student roles
- Secure API access

Core Features

- User Management System
- Grades Management Module
- Pagination and filtering support
- Grade publishing system
- Automatic total grade calculation

Business Rules

- Prevent deleting doctors assigned to courses
- Validation for grade limits
- Enforcing domain integrity

Design Patterns

- Repository Pattern
- Dependency Injection
- DTO Pattern

Backend Architecture & Implementation

Error Handling

- Global exception handling middleware
- Standardized API responses
- Clean frontend-backend integration

API Capabilities

- RESTful APIs
- Secure JWT access
- Pagination & filtering
- Structured error responses

Backend Advantages

- Modular and testable architecture
- Clear separation of concerns
- Scalable backend system
- Real-world software engineering practices
- Centralized error handling
- Strong maintainability and flexibility

AI Module
(Student Performance Analysis)

Weighted Scoring (Average) Algorithm

Each student gets a score based on:

- Attendance (30%)
- Assignments (25%)
- Quiz (35%)
- Trend (10%)

Rule-Based Risk Classification

Students are classified into:

- Low Risk
- Medium Risk
- High Risk

Based on rules (attendance, quiz, score)

Failure Probability

The system estimates failure probability:

- Score ≥ 75 → Low Risk → Failure Probability = 10%
- Score ≥ 60 and < 75 → Medium Risk → Failure Probability = 40%
- Score ≥ 50 and < 60 → High Risk → Failure Probability = 60%
- Score < 50 → Very High Risk → Failure Probability = 80%

AI Module
(Student Performance Analysis)

AI Processing Flow

1. Receive student data from API
2. Validate data using Pydantic
3. Calculate weighted score
4. Determine risk level
5. Estimate failure probability
6. Return analysis results

Technologies & Concepts

- Python-based AI logic
- Pydantic for data validation
- Rule-Based Decision System
- Weighted Scoring Algorithm

Advantages

- Simple and fast implementation
- Easy to explain and understand
- No large datasets required
- Easy to maintain and improve

Limitations

- Uses fixed rules
- Does not learn from data
- Less accurate than Machine Learning models

Future Improvements

- Integrate Machine Learning models
- Improve score weights using real data
- Add self-learning capabilities
- Enhance prediction accuracy

Flutter Mobile Application

Architecture Flutter Mobile Application

Architecture Pattern: Feature-Based Clean Architecture

The project implements a modern feature-based architecture inspired by Clean Architecture principles.

This approach provides:

- Scalability
- Easy to add new features
- Maintainability
- Clear separation of concerns
- Testability
- Independent feature testing
- Reusability
- Shared core utilities

Architecture Layers

PRESENTATION LAYER
(Views, Widgets, Managers)
DOMAIN LAYER
(Business Logic, Entities)
DATA LAYER
(Models, Services, Repositories)
CORE INFRASTRUCTURE
(Network, Utilities, Assets)

Mobile Application Guide

Architecture & Development

- Clean Architecture with feature-based modular structure
- Scalable and maintainable codebase
- Separation of concerns between layers
- Reusable components and custom widgets
- Cross-platform development using Flutter & Dart

Authentication & Security

- JWT Authentication system
- Role-Based Access Control (Student / Doctor)
- Secure API communication
- Input validation and error handling
- Protected APIs and token management

API Integration

- REST API integration using Dio
- Centralized networking layer
- Authorization headers handling
- Async requests and exception management
- Real-world backend communication

Academic Features

- Course Management System
- Quiz & Exam System
- Assignment Management
- Attendance Tracking
- Grades & Results System
- Schedule & Lecture Management

AI Features

- AI Assistant powered by Google Generative AI
- Homework help and content summarization
- Smart chatbot integration
- Learning recommendations

Mobile Application Guide

UI/UX Design

- Responsive and adaptive UI
- Modern and organized design
- Smooth navigation experience
- Reusable UI components
- Multi-platform support

Performance & Optimization

- Provider optimization using Riverpod
- Reduced widget rebuilds
- Localized state management
- Feature separation for better scalability
- Optimized application performance

Team Workflow

- Team collaboration using Git & GitHub
- Branch-based development workflow
- Modular feature development
- Organized project structure

Future Improvements

- Google & Facebook Authentication
- Push Notifications & Real-Time Chat
- Dark Mode & Localization
- Firebase Integration
- Offline Support & Caching
- Advanced Analytics & Reports

Mobile Application Guide

Features

****Authentication & Authorization****

Email/Password-based login system - User registration with faculty selection - Role-based access control (Student/Faculty) - JWT token-based authentication - Secure token storage and management - Password validation with

security best practices Course Management

Browse and search available courses - Course enrollment system - Course descriptions and syllabus - Faculty information and contact details - Course prerequisites and requirements - Batch-wise course organization

Quiz & Exam System

For Students: View available quizzes - Real-time quiz attempt with timer - Question progress tracking - Instant quiz submission - View quiz results and score analysis - Review quiz

answers and explanations

For Faculty: Create and manage quizzes - Add multiple-choice and descriptive questions - Set quiz duration and deadlines - View student submissions and analytics - Grade quiz

responses Assignment Management

Assignment creation and publishing - File upload support for submissions - Deadline tracking with visual indicators - Submission status tracking - Faculty feedback system - Grade display and history

Grades & Results System

Comprehensive grade tracking - GPA calculation - Performance analytics - Semester-wise grade history - Downloadable grade reports - Subject-wise performance breakdown

Mobile Application Guide

Schedule & Lecture Management

- Weekly class schedule view - Lecture timing and location information - Room/Hall assignment details - Calendar integration preparation - Lecture notes download - Session-based organization

Attendance System

QR code-based attendance marking - Manual attendance recording by faculty - Attendance history tracking - Attendance percentage calculation - Absence notifications - Attendance reports

News & Events

University announcements and news feed - Upcoming events calendar - Event details and registration - Notifications for important events - News categorization

User Profile Management

Personal information display - Academic details and statistics - Profile picture upload - Enrolled courses overview - Contact information management - Personal settings

AI Assistant Features

Powered by Google Generative AI - Study material summarization - Homework help and explanations - Real-time chat interface - Document analysis - Learning recommendations

Onboarding System

Welcome screens for new users - Feature introduction - Role selection guidance - Quick setup wizard - Tutorial system

Real-time Dashboard

Personalized student dashboard - Quick access to courses and assignments - Upcoming deadlines widget - Grade overview - Recent activities

Dashboard

Key Features

- Secure login page with username and password authentication.
- User management with add, edit, and delete functionality.
- Role-based system supporting Students, Doctors, and Admins.
- Faculty and department management.
- Course catalog with full CRUD operations.
- Student enrollment system.
- Results management with faculty filter and student search.
- Student transcript viewer.
- Events and News section with tab switching.
- Responsive design supporting desktop and mobile.
- Single Page Application (SPA) with no page reloads.

Challenges Solved

- Built a fully functional admin panel using only vanilla JavaScript without any backend.
- Implemented dynamic filtering and search across multiple sections.
- Designed a clean and modem Ul using Tailwind CSS utility classes.
- Organized JavaScript code into separate modular files for maintainability.

User Input & Prediction Workflow

Student Inputs	Doctor Inputs	Admin Inputs
Quiz answers	Grades	User management
Assignment submissions	Quiz creation	Course management
Attendance records	Assignments	System monitoring
Course interactions	Attendance tracking	Publishing Grades

Prediction Workflow:

1. Student data is received through APIs
2. Data is validated using Pydantic
3. The score is calculated using weighted scoring
4. Risk level is determined
5. Failure probability is predicted
6. Results are displayed on Dashboard and Mobile App

Challenges & Solutions

Challenges:

- API integration
- State management
- Authentication handling
- Dynamic filtering
- Organizing modular architecture

Solutions:

- Riverpod for state management
- Clean Architecture
- JWT Authentication
- Reusable components
- Service-based structure

Future Work

AI Improvements

1

- Machine Learning integration
- Deep Learning models
- Personalized recommendations

Backend Improvements

2

- Cloud deployment
- Real-time communication
- Microservices architecture

Mobile App Improvements

3

- * GPA Calculator
- * Assignment Submission Uploads
- * Quiz Analytics
- * Attendance Reports

Conclusion

This project presents a complete smart academic management system that integrates Backend, AI, Dashboard, and Flutter mobile application technologies into one centralized platform.

The backend system was designed using Clean Architecture and ,Domain-Driven Design (DDD) principles to ensure scalability maintainability, security, and clear separation of concerns. It provides ,secure RESTful APIs, role-based authentication, user management course management, grading, attendance tracking, and academic .operations handling

The AI module adds an intelligent layer to the system by analyzing -student performance using weighted scoring algorithms and rule based classification. It helps detect at-risk students early, estimate failure probability, and support academic decision-making through .simple and explainable logic

The dashboard provides administrators with a modern and interactive ,interface for monitoring university data, managing users and courses visualizing analytics, filtering results, and tracking student .performance efficiently

In addition, the Flutter mobile application delivers a responsive and ,user-friendly experience for students and doctors. It supports quizzes assignments, attendance, grades, schedules, events, and news while maintaining scalable architecture using Riverpod and feature-based .structure

Overall, the project demonstrates how modern software engineering -practices, clean architecture principles, AI techniques, and cross ,platform mobile development can be combined to build a scalable maintainable, and intelligent academic management solution suitable .for real-world educational environments and future enhancements

References

- Microsoft ASP.NET Core Documentation (Microsoft)
- Entity Framework Core Documentation (Microsoft)
- Microsoft SQL Server Documentation (Microsoft)
- FastAPI Documentation (Open Source Project by Sebastián Ramírez)
- Tailwind CSS Documentation (Tailwind Labs)
- JavaScript MDN Documentation (Mozilla Developer Network – Mozilla Foundation)
- JWT Authentication Documentation (Auth0 / Open standards – IETF RFC 7519)
- Flutter Documentation (Google)
- Dart Documentation (Google)